EXPERIMENT 11

Shravanya Andhale

D20A-02

Deployment of a Generative AI Application Using Gradio/Streamlit

AIM:

To develop and deploy an interactive Generative AI application using a pretrained Large Language Model (LLM) and a Gradio or Streamlit user interface, demonstrating model loading, prompt handling, text generation, user interaction, output control, and basic application deployment.

THEORY:

Generative Artificial Intelligence applications use pretrained models to generate new content based on user instructions or input data. Large Language Models are an important part of Generative AI because they can understand natural language prompts and generate meaningful responses for tasks such as question answering, summarization, text generation, code assistance, and conversational applications.

In this experiment, a pretrained LLM is integrated into an interactive application using Gradio or Streamlit. The model is not trained from scratch. Instead, an already trained model is loaded and used for inference, which means generating responses using its existing learned parameters.

1. Pretrained Large Language Models:

A pretrained LLM is a neural network that has already learned language patterns and relationships from a large text corpus. It can perform different generative tasks through prompting and inference. Depending on the selected model, it can be accessed through a local model library, model repository, API, or institutional service.

The exact model name and version must be recorded. The selected model should be tested independently before integrating it with the user interface.

2. Generative AI Application Architecture:

A Generative AI application connects user input with a pretrained model and displays the generated response. The application generally consists of the following stages:

1. User Input: The user enters a question, instruction, or prompt through the interface.

2. Input Validation: The application checks whether the input is empty, too long, or invalid.
3. Prompt Construction: The input is combined with system instructions, task requirements, and formatting constraints.
4. Model Inference: The constructed prompt is passed to the pretrained LLM.
5. Text Generation: The model generates a response based on the prompt and its learned parameters.
6. Output Processing: The generated response can be checked or formatted before presentation.
7. Output Display: The final response is displayed through the application interface.

3. Interactive User Interface:

Gradio and Streamlit are Python frameworks used to develop interactive web applications for machine learning and data science projects.

Gradio provides components such as textboxes, buttons, dropdowns, sliders, and output areas. It allows developers to connect a Python function with user interface components and quickly create an interactive machine learning application.

Streamlit is another Python framework used to build interactive data and AI applications. It provides widgets such as text inputs, buttons, sliders, and selection controls that allow users to interact with Python programs through a web interface.

Both frameworks can be used to create text-generation or chatbot applications.

4. Prompt Engineering:

Prompt construction is an important part of a Generative AI application. A system prompt can define the role and purpose of the application, while the user prompt contains the actual question or instruction. The prompt may also specify the expected response format, domain restrictions, and desired level of detail.

Different prompt versions can be tested to identify which produces the most relevant and useful responses. The model should also be instructed to state uncertainty rather than invent unsupported information.

5. Generation Parameters:

LLM applications may expose generation settings to control the output.

- Temperature: Controls the randomness of token selection. Lower values generally produce more consistent outputs, while higher values can produce more varied responses.
- Maximum New Tokens: Limits the maximum length of the generated response.

- Top-p: Controls token sampling by considering a group of likely tokens whose cumulative probability reaches a specified threshold.

These parameters influence response diversity, consistency, and length. Their availability depends on the selected model.

6. Inference and Training:

Training involves adjusting the model's weights using data so that it learns patterns. Fine-tuning further adapts an existing model to a specific task or domain. Inference, on the other hand, uses the already trained model to generate outputs without normally changing its weights.

This experiment focuses on inference rather than training or fine-tuning, making it possible to build an application without the computational resources required to train a large language model.

7. Testing and Responsible Use:

The application should be tested using different prompts to check response quality, relevance, usability, and reliability. Empty inputs, invalid requests, and unexpected errors should be handled appropriately. The application should also include suitable limitations and responsible-use information because LLM responses may contain incorrect or unsupported content.

The final application demonstrates how pretrained language models can be integrated into practical user-facing systems through Python-based interface frameworks.

CODE AND OUTPUT:

Experiment 11 - Interactive Generative AI Application using Gradio

```
!pip -q install transformers sentencepiece gradio torch
```

```
...
```

```
import torch, gradio as gr
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
MODEL_NAME="google/flan-t5-small"
device="cuda" if torch.cuda.is_available() else "cpu"
tokenizer=AutoTokenizer.from_pretrained(MODEL_NAME)
model=AutoModelForSeq2SeqLM.from_pretrained(MODEL_NAME).to(device).eval()
print("Loaded",MODEL_NAME,"on",device)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
config.json: 100% 1.40k/1.40k [00:00<00:00, 39.6kB/s]
tokenizer_config.json: 100% 2.54k/2.54k [00:00<00:00, 127kB/s]
spiece.model: reconstructing file: 100% 792kB / 792kB, 77.4kB/s
spiece.model: downloading bytes: 649kB, 63.4kB/s
tokenizer.json: 100% 2.42M/2.42M [00:00<00:00, 16.5MB/s]
special_tokens_map.json: 100% 2.20k/2.20k [00:00<00:00, 21.9kB/s]
model.safetensors: reconstructing file: 100% 308MB / 308MB, 28.7MB/s
model.safetensors: downloading bytes: 291MB, 26.5MB/s
Loading weights: 100% 190/190 [00:00<00:00, 2657.01it/s]
[transformers] The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.weight, but both are present in the checkpoints with different values, so
generation_config.json: 100% 147/147 [00:00<00:00, 10.3kB/s]
Loaded google/flan-t5-small on cpu
```

```
def generate_response(user_input, temperature=0.7, max_tokens=128, task="Coaching suggestion"):
    if not user_input or not user_input.strip():
        return "Please enter a question or training goal."
    if len(user_input.strip()) > 1200:
        return "Please shorten your input to 1,200 characters or fewer."
    prompt = (
        "You are a careful, supportive cricket batting practice assistant. "
        "Give practical, beginner-friendly guidance. Do not claim to analyze video or measure a player's body. "
        "Do not guarantee match outcomes or diagnose injuries. If evidence is insufficient, say so. "
        f"Task: {task}. User input: {user_input.strip()} "
        "Answer clearly using short bullet points where helpful."
    )
    inputs=tokenizer(prompt,return_tensors="pt",truncation=True,max_length=512).to(device)
    do_sample = float(temperature) > 0.05
    kwargs={"max_new_tokens":int(max_tokens),"do_sample":do_sample}
    if do_sample:
        kwargs.update({"temperature":max(0.1,float(temperature)),"top_p":0.9})
    with torch.no_grad():
        output=model.generate(**inputs,**kwargs)
    response=tokenizer.decode(output[0],skip_special_tokens=True).strip()
    return response or "The model did not return a response. Try rephrasing the question."

# Smoke test before creating the UI.
print(generate_response("Suggest a short warm-up before batting practice.",0.4,80,"Practice plan"))
```

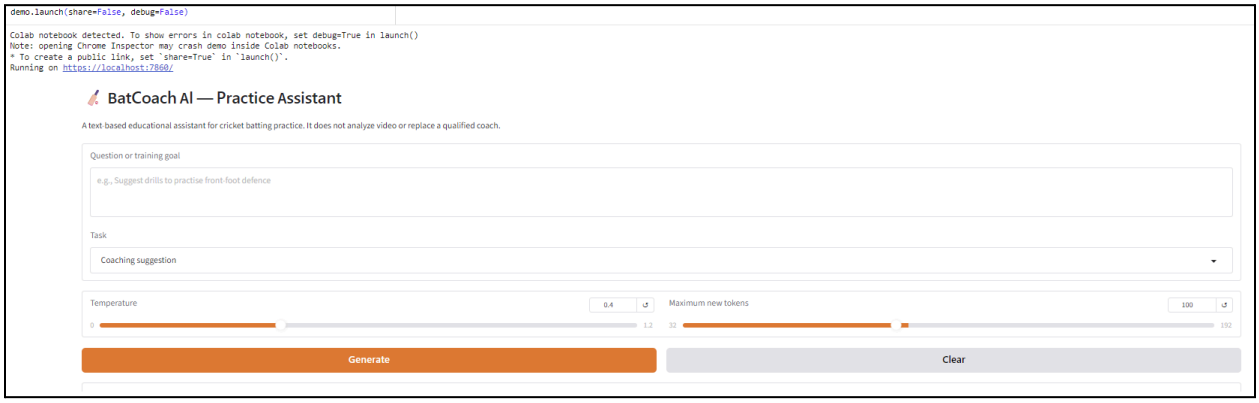
```
Choose a warm-up.
```

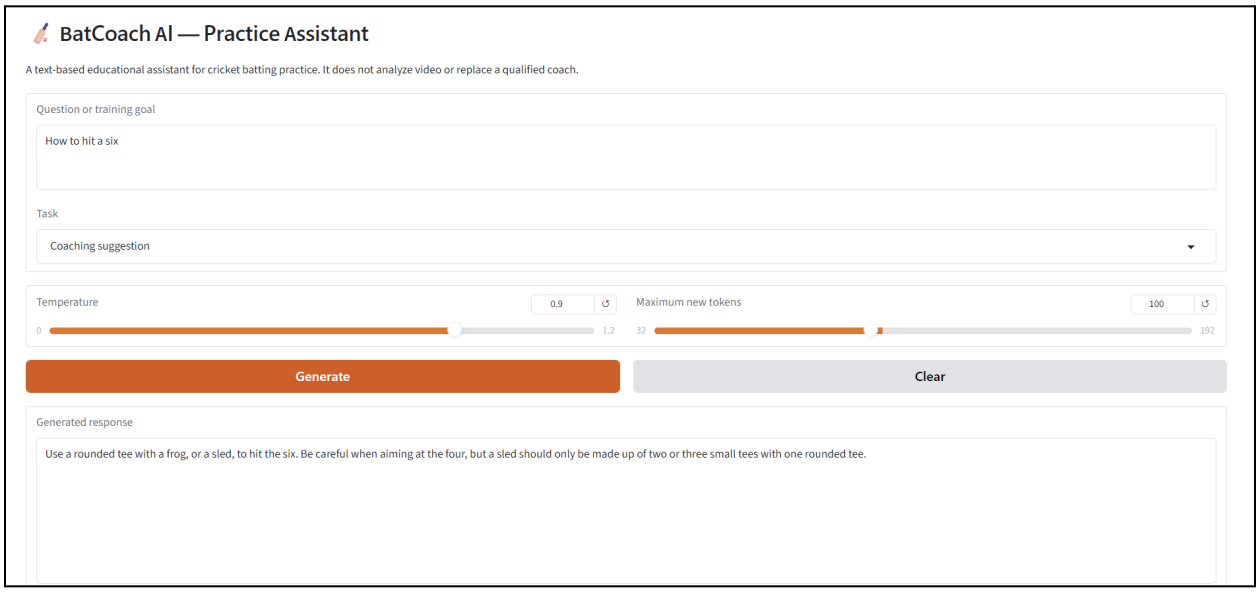
```python
with gr.Blocks(title="BatCoach AI — Practice Assistant") as demo:
    gr.Markdown("# 🏏 BatCoach AI — Practice Assistant")
    gr.Markdown("A text-based educational assistant for cricket batting practice. It does not analyze video or replace a qualified coach.")
    question=gr.Textbox(label="Question or training goal",placeholder="e.g., Suggest drills to practise front-foot defence",lines=3)
    task=gr.Dropdown(
        choices=["Coaching suggestion","Practice plan","Explain a technique","Summarize a training concept"],
        value="Coaching suggestion",label="Task"
    )
    with gr.Row():
        temperature=gr.Slider(0.0,1.2,value=0.4,step=0.1,label="Temperature")
        max_tokens=gr.Slider(32,192,value=100,step=16,label="Maximum new tokens")
    with gr.Row():
        submit=gr.Button("Generate",variant="primary")
        clear=gr.ClearButton([question,task,temperature,max_tokens])
    output=gr.Textbox(label="Generated response",lines=8)
    submit.click(fn=generate_response,inputs=[question,temperature,max_tokens,task],outputs=output)
demo
```

```
Gradio Blocks instance: 1 backend functions
------------------------------------------
fn_index=1
 inputs:
 |-<gradio.components.textbox.Textbox object at 0x78bde8d27770>
 |-<gradio.components.slider.Slider object at 0x78bde8d27b60>
 |-<gradio.components.slider.Slider object at 0x78bde8cfae90>
 |-<gradio.components.dropdown.Dropdown object at 0x78bde8d278c0>
 outputs:
 |-<gradio.components.textbox.Textbox object at 0x78bde8cfb110>
```

```
demo.launch(share=False, debug=False)
```

```
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
Note: opening Chrome Inspector may crash demo inside Colab notebooks.
* To create a public link, set `share=True` in `launch()`.
Running on https://localhost:7860/
```





CONCLUSION:

The experiment demonstrated the development of an interactive Generative AI application using a pretrained Large Language Model and a Gradio interface. The application connected user inputs to model inference and displayed generated responses through an interactive interface. Prompt construction, generation controls, input validation, and error handling helped improve the application's usability. The experiment provided practical understanding of deploying pretrained LLMs in real-world applications without training a model from scratch.